Delta Gym Management System

Project Report

CST2550- Software Engineering Management and Development

Team Members:

Jay Yang (M01006234)

Aththinna Marakkalage Pasindu Pansilu Buddhadasa (M01007521)

Hesandu Siriwardana (M01018921)

Noeyal Batolar (M01087484)

Samrad Klidsar (M00999775)

Submission Date: April 19th, 2026

Contents

1. Introduction
    a. Project Overview
    b. Report Structure
2. Project Management
    a. Team Roles & Responsibilities
    b. Agile Methodology & Team Management Tools Used
    c. Sprints, Stand-ups, and Meeting Evidence
3. System Design
    a. Design Overview
    b. User Requirements
    c. Data Model Design
    d. Architectural Design
    e. Algorithm Design and Analysis
    f. Frontend Design
4. Implementation
    a. Database Integration
    b. Identity and Authentication
    c. Core Features
5. Testing
    a. Testing Approach
    b. Build Verification
    c. Functional Testing
    d. Challenges Encountered
6. Conclusion
    a. Summary of Work
    b. Critical Reflection
7. References

Introduction

Project Overview

Delta Software was assigned to create a Gym Membership management system. As assigned, they endeavored to create a Gym Membership Management system to facilitate the creation, management, and deletion of gym memberships. However, Delta Software has taken it upon themselves to expand upon simple membership management, and created a system designed for nearly all facets of gym operations. While Delta has successfully created a system that manages memberships, and their related subscriptions and payments, they have also included such areas as, but not limited to, staff, trainer, booking, and equipment management. The purpose of this is to address the problem of needing multiple software, when one piece of software can do it all. As opposed to being defined as a Gym Membership application, a more apt description of this application is that of this: a Gym Management application.

So, it can be surmised that the Delta Gym Management System is a full-stack, progressive web application (PWA) developed to support the everyday running of a gym. The purpose of the system is to make it easier to manage key gym operations such as members, staff, trainers, bookings, sessions, attendance, schedules, subscriptions, payments, branches, and rooms from a single platform. The project was built using ASP.NET Core with a separated Application Programming Interface (API) and Model-View-Controller (MVC) consumer structure, which allowed the frontend and backend parts of the system to be organized more clearly. Different users can access different parts of the application depending on their role, which makes the system closer to a realistic gym management platform.

Report Structure

The primary aim of this report is to discuss the design, architecture, and implementation processes taken by the Delta Software team to develop this application. There will be an attempt to document the development of the application with a chronological mindset; however, there will be divergence to speak about specific matters when they are most meaningful. Due to the limit applied to this report's length, only the most important parts of each section will be discussed, as there is a substantial amount of information to relay. This report is organized into sections covering project management, system design, implementation, testing, conclusion, references, and appendices. Discussion will begin with describing how the project was managed and how the team worked together, and then explain the overall system design and architectural choices. After that, the report will detail implementation and testing of the codebase before concluding with, and making further note of, the most important achievements made. Any mention to evidence and related files are towards the included files found in the repository.

Project Management

Team Roles & Responsibilities

At the start of this endeavor, there was a period of time taken to decide which member of the team would take on which role (other than the team lead). By assessing the knowledge, determination, and past work of each individual, it came to pass that the roles of each member would be of those that were supplied to The Syndicate. Naturally, each member would have to take part in work pertinent to each role, as five members is too small a force for the task. However, each individual would spend most of their time and effort in work related to their specific role. At least, that was the initial plan; there is a desire to be honest and true. What happened instead was more on the line of improvisation, and common sense, as absenteeism and lack of response to communication attempts required, and caused, a deep shift in the team dynamic and handling of responsibilities. For a brief moment in time, only the team lead was actively working on the project. So, when it comes to responsibilities, the biggest thing to note is that the team lead took the brunt of it. When it came to the point when more people were willing to work, the immediate tasks were sorted appropriately, though at times the responsibility would have to be picked up due to time constraints.

Agile Methodology & Team Management Tools Used

An Agile approach was attempted. Work was completed iteratively, with features being added, tested, reviewed, and improved over time. However, the team could not follow the traditional Scrum ceremony. This is due to the aforementioned lack of "showing up" and, quite frankly, a great misunderstanding and lack of communication from The Syndicate. As such, it was extremely difficult to have continuous improvement that adhered to Agile methodologies accurately. During these sprints of development, the team added features in small stages, iteratively testing and fixing as the project came along.

Discord was used as the main communication platform for meetings, work reporting, progress updates, and issue discussion. This is due to its high accessibility, being available as a desktop, mobile, and browser-based application while not needing payment for access. It features calling, screen sharing, file-uploads, and organization. GitHub was the choice when it came to hosting the repository. GitHub is, easily, the most accessible and familiar choice when it comes to hosting a repository. It is well known, well supported, and well documented. When utilizing branch protection rules, it was infinitely easier to ensure that no accidental rewrites or pushing of dangerous code could happen. The team followed a branching strategy that is completely laid out on the application repository's README file.

Sprints, Stand-ups and Meeting Evidence

As stated previously, there was an attempt to adhere to Agile methodology. That being said, it must be stated that meetings and stand ups came with great difficulty. The design itself took seven weeks to complete. Though there were some meetings, around three to five, during this, meetings did not become consistent until week five. Even then, there were presence issues. However, starting from week five, there was a physical meeting every Tuesday and stand-ups on Thursdays and Saturdays. Physical meetings consisted of discussing tasks, application architecture, git workflow, and general development processes. They served as an opportunity for those who were struggling or confused about the project to share their questions and problems, and served as an opportunity to discuss ideas and improvements to the application. Any such evidence towards these meetings will be provided, along with a comprehensive project log.

System Design

Design Overview

The approach Delta Software took in the design of the application followed a hybrid of waterfall and Agile techniques. It was known that the design process would consist of: defining use cases, defining user requirements, data design, architectural design, and backend design. As such, an interactive approach was taken with the use cases and user requirements of the system, where each individual's ideas were consolidated into a single, well-defined document for all to reference. Designing entities was a necessary step in understanding relationships between the different parts of the application. It also helped in understanding how interaction with the application should occur. In essence, the data needed to be understood before proper dictation of the application's architecture. Defined entities and entity relationships facilitated the discussion of what specific format/architecture could be used to create, retrieve and update that data. Following that, ideas for frontend design, and specific interaction flow, were easier to draft. In essence, each step of the process aided in producing the next.

Use Cases & User Requirements

As stated, the first phase of the design process began with generating use cases for all aspects of the software. This included goals such as purchasing a membership, signing up for an account, booking a class, and, essentially, all of the important features that should be available through our system. To view each individual team member's specific use case generation, please refer to supplied files. The most notable, and important aspects of each use case were collated into a single use case document. From this, Delta Software were able to accurately formulate a list of user requirements, organized by function and role; also found in the supplied files.

Data Model Design

One of the most overlooked parts of the design process by up-and-coming developers is that of data modeling. To immerse oneself with the data they intend to be working with, to understand how everything connects, makes designing and developing a system exceptionally easy, logical, and clear. An effort was made to design a relational diagram to increase the clarity in what features are expected in the system. There were clear, obvious entities from the beginning. However, as the specification of gym membership management shifted to gym management, the need for more entities grew as well. Of the obvious entities, the most important were the member, tier, subscription, and payment entities as they define the "membership management" part of the system. Being more than this, however, the system must also cover personnel, infrastructure, operations, and gym access. The Staff, Trainer, and Admin entities cover personnel, which is crucial for understanding access levels and the different employees in the system. The Branch, Room, and Equipment entities cover infrastructure, which is important to understanding the physical space where operations occur. The Class and Session entities cover operations, which is important to understanding the on-grounds services supplied by the gym. Finally, the Attendance entity covers gym access, which is important for understanding how allowance to gym facilities is issued. The make-up of each entity, each property, aids in defining the records within them with accurate and necessary information relative to the gym management system.

Architectural Design

Designing entities and entity relationships is evidently important in the design of systems. However, equally important is the interface, or instrument, with which to handle and display that information. There were a few key options: a simple command-line interface (CLI), a WinForm desktop application, or a web application. If it were the case that the system were a simple, in-house application, then a simple CLI would suffice. The users of the software, however, will not be solely employees or administrative staff. Thanks to the efforts made in

designing the database, the team was able to understand the user pool associated with the system. And considering that the system would be used by different roles, across different hardware, it made the most sense to create a web application for seamless, immediate access across multiple devices at all times. As such, the architectural strategy involves a decoupled two-project solution, an arrangement engineered to delineate concerns between data persistence and user interaction. In essence, the backend comprises a RESTful API tasked with the governance of complex entity structures, while a contemporary ASP.NET MVC application serves as the primary consumer for the client interface. Naturally, this separation necessitated a shared class library to facilitate seamless serialization; by centralizing Data Transfer Objects (DTOs) and immutable records within this common assembly, rigorous type safety across the network boundary is ensured. Of course, this decoupled nature allows the API to be consumed by more than a website. This can be expanded to a mobile app or desktop application in the future, as long as they communicate with the API using the same library.

Algorithm Design & Analysis

From the beginning, there was a desire to mitigate any issues in terms of efficiency and scalability. As such, the system's foundational data operations are characterized by a pragmatic approach to scalability. Due to the hard limit on this report, any such pseudocode will be supplied in the relevant directory of the repository's files. For that same reasoning, this section of the report will only cover the designing of the most pertinent algorithmic behaviors within the Delta Gym System codebase. These such algorithms and pattern designs relate to: data retrieval and search operations, cryptographic and interactive systems, and transactional logic.

Naturally, querying infrastructure serves as the backbone of the application. It can be assumed that such algorithms will be the most resource heavy operations of the system. The pagination mechanism relies on a complete row count, culminating in an O(n) time complexity as the workload increases with the number of the inputs linearly, considered a persistent overhead. This pagination goes hand-in-hand with the suggested search functionality of the design process. The multi-field text search introduced a particularly heavy performance bottleneck. The operations in the design directly map to the SQL "LIKE '%term'" with leading wildcards, which causes indexing to be entirely bypassed and necessitated full table scans. It was assumed that this would yield an O(n x m) complexity, where n is the number of rows, and m is the number of searched columns. During review, Delta struggled with balancing these robust search features against database strain. Ultimately, a dynamic sort dispatch was implemented to facilitate better query execution. By pattern-matching the targeted column in constant time, and passing on the O(n log n) sorting burden directly to the database layer, the architecture justifies its constraints by relying on native SQL optimizations.

In comparison, the security layer exhibits commendable efficiency. The JWT Generation and HMAC-SHA256 Token Signing algorithms both operate at O(1) complexity, as the computation time is tied to fix-length payloads. Adding secure features and AI tools was a steep learning curve for team Delta. For example, generating QR codes, which uses the HMAC-SHA256 Token Signing algorithm, can take a lot of processing power, leading to an O(n2) if the QR Code isn't saved (cache miss). It was immediately evident that fresh generations for every request would crash the system. As such, generated codes were cached, bringing the average time down to O(1), amortized. This required extensive research into the caching solutions offered in ASP.NET. For agentic features, the LLM Tool Loop algorithm introduces a variable, yet bounded, complexity. The loop dynamically routes user prompts to internal endpoints with an O(n) complexity per tool execution. There was initial unfamiliarity with unbounded non-deterministic generative models that forced honest reflection about the risks of infinite execution loops. To mitigate this, and maintain system stability, this was constrained to a maximum of six rounds. The payment and booking transactions rely on strict, step-by-step checks. The booking process checks for duplicate entries by scanning the session's current bookings; resulting in an O(n) operation. The subscription queueing, however, finds

the latest end date of active plans to figure out billing, which is a perceived O(n) complexity that is approximately (≈) O(1). These methods were chosen specifically to ensure that data is always accurate and to ensure safety rather than speed. It is clear that while the O(1) security operations provide a stable foundation, the O(n) search patterns represent the most significant fault to be addressed in future iterations. Further details can be found in the pseudocode file.

Frontend Design

In the process of frontend design, we as a group had discussions on how we can approach the structure of the pages, colour palettes, fonts, and a lot more elements, since the website is separated into 3 parts/areas including Members, Trainers, and Management, we took different approaches and ideas for all the sections for example, in members we needed engaging and user-friendly design however, in the management or trainer areas, we focused more on the information, functionality, and accuracy.

Each person who was in charge of the frontend, came up with different design ideas, and through discussion we chose one design, efforts were also made to create design ideas on Figma which helped us gain a better understanding of the final product.

In our member section, we decided to choose a design that is more minimalistic and modern. A simple design avoids confusion and overload of information, and the user is more likely to engage with the website. That being said, necessary and important details are provided, so that the user won't feel there is a lack of information. As for other areas and sections of the website such as trainer and management, we focused more on the functionality rather than design however we made sure it is convenient for the users to work on those pages as well.

While working on coding with the word of everyone we had to take out some from the first design and change some elements and new pages so here's the Figma design that we worked on and the flow:
https://www.figma.com/design/hqrL1IsBl3Mga3vBXfEV7d/DELTA?node-id=0-1&t=vMvXCdx44XVi7OMx-1

Moreover, in the implementation stage of the project, which includes creating the actual front end and connecting it to the backend, we took different approaches, we began by creating them on our separate individual projects as this was more convenient for us to focus more on the design aspect rather than the functions or wiring, shortly after that, we implemented our project in our own branches on github, and later on was merged together.

To avoid conflict merging, we all worked on the separate folders/areas. However, since backend was still in development and improvement, we made our best effort to keep our version of the project updated as this could have caused potential conflict merges.

Implementation

Database Integration

Let it be known that Delta Software took a 'code first' approach towards the creation and definition of the database and its schemas. As such, the API utilizes Entity Frame (EF) Core atop Structured Query Language (SQL) Server, configured with connection pooling and transient-fault retry logic; with an acknowledgement that database connectivity, particularly in cloud-hosted environments, is not without faults. In addition to this, however, the usage of ASP.NET Identity is also incorporated for ease of authentication and role- based access levels. It is because of this that the database context (GymDbContext) of the system inherits directly from Identity's database context (IdentityDbContext). This mixes in the application's custom tables (Tiers, Branches, Classes, Rooms, Equipment, Sessions, Schedules, Subscriptions, Bookings, Payments, and Attendances) with built-in Identity tables. A single 'ApplicationUser' model represents all personnel types; Members, Staff, Trainers, and Admins; differentiated solely by their assigned Identity role. This was a deliberate design choice, and was always the intention since the design phase. Separating by role was favored over fragmentation of the user hierarchy across multiple tables, as a unified model facilitates simpler querying and relationship mapping, albeit at the cost of nullable fields that pertain only to certain roles.

Relationships are explicitly configured in the OnModelCreating method of the application database context using EF Core's native Fluent API, with careful attention to cascade behaviors. One such relationship is the Payment-to-User relationship setting the delete behavior to "no action" to circumvent the multiple cascade path conflict that would otherwise arise from its parallel relationship through Subscription. In addition, the database is seeded on startup, which ensures that the four roles; members, staff, trainers, and admins; exist in the system.

Identity and Authentication

As stated, the system uses ASP.NET Core Identity, as there was a desire to not build authentication infrastructure from the ground up. Identity's base "user" model was customized with properties pertinent to the system, and is configured with deliberate rigor, with rules imposed on passwords and usernames. In addition, thanks to its native behavior, unique emails are enforced at the Identity level. Authentication, itself, is stateless; JavaScript Object Notation (JSON) Web Tokens (JWT) are issued by a custom token service. Upon successful login, the service constructs a token containing claims for the user's database identifier, email address, given name, surname, and all assigned roles. As designed, the JWT token service uses an HMAC-SHA256 signing algorithm. The JWT Bearer middleware enforces validation on issuer, audience, lifetime, and signing-key parameters. Naturally, stateless tokens present a revocation challenge; when JWTs persist as valid even when the issue-ee has ceased usage (i.e. logged out of the system). As such, each token also carries a unique JWT ID (JTI) claim. Upon logout of the system, the specific JTI is added to a blacklist (in-memory cache). This is vital for security of the system, as any such request containing a blacklisted JTI is invalid, and access is denied.

Core Features

In fear of further going over the limit enforced on this report, the core features of the system will be consolidated into this singular section, as information and discussion of present features may potentially reach the limit of seven pages in itself. As such, bear in mind that discussion of each feature will be kept extremely brief to adhere to this; there will be more "what" and less "why". The core features of the system are Management with Search and Pagination, Attendance and QR Code Scanning, Memberships and Subscriptions, and Agentic AI Chatbot Integration.

Considering that this is a gym management system, it is evident that endpoints and an

interface was necessary to employ Create-Read-Update-Delete (CRUD) operations. As such, a management dashboard was created for Staff and Admin roles, where simple CRUD operations on the various models, personnel, and infrastructure could be made. To facilitate this, and not have to sift through an unknown number of records, search, filter, and sort features were implemented, along with pagination. A search term is applied as a case-insensitive substring match across pertinent text fields, such as email; subject; address, and uses EF Core's Language Integrated Query (LINQ) to SQL for translation. Sorting is implemented via pattern-matched switch expressions that map string identifiers to OrderBy/OrderByDescending clauses. A paged result extension method clamps the page size between one and one-hundred, counts total matching rows, applies skip/take logic, and wraps the results in an object envelope containing the items alongside relevant metadata.

As it was stated, attendances can be manually managed using basic CRUD operations via a management dashboard. Members can simply view their attendance history. The most prominent part of this attendance management is the introduction of automation. Naturally, a real-world gym encounters the scenario wherein a member may neglect to scan out. A hosted background service is in place to rectify this by executing a sweep every sixty minutes. It queries for attendance records whose status is marked as "in" and check-in timestamp exceeds a four-hour threshold. The background service programmatically sets the check-out time and clears the status of each record that matches the query. The most pragmatic solution, it prevents phantom occupancy from accumulating indefinitely.

As designed, Delta Software was aware of the need for some sort of member attendance feature, apart from the simple CRUD operations that could be performed by staff. Evidently, the solution came to be check in's and check out's with an issued QR code. The QR code mechanism is arguably the most distinctive operational feature of the system. There are two phases to consider: generation and scanning. The system produces a time-limited QR code image (Base64) for a given member from a payload of that member's ID and an expiry date timestamp. When a QR code is scanned, it is passed to, and checked by a validation method that checks if the decoded Base64 code is valid and has not expired. If it is valid and timely, the system queries for the attendances for an open session for that particular member. The presence of an open session triggers a check-out, while absence of an open session triggers a check-in.

The subscription lifecycle is one of the more architecturally involved features, managed by a four-state enumeration: Pending, Active, Queued, and Expired. When a subscription is first created, whether by Staff or a Member, it enters the Pending state with defaulted start and end dates. It remains in this state until a payment is processed. This deliberate decoupling was a design decision rooted in the desire to model the real-world flow faithfully; where a member selects a tier and the subscription is created only upon financial settlement. As such, payment serves as the activation mechanism. If the target subscription is Pending, the method transitions it to Active, stamps the current time as the start date, and sets the end date to one month later. It also activates the member's account if it was previously inactive. If, however, the target subscription of that member is already Active or Queued, the system creates a new Queued subscription whose start and end dates that directly trail the latest subscription by one month respectively. Initially, the Stripe API was being heavily considered to aid in the matter of subscription handling, but in efforts to reserve the time spent learning the API for something else, this was chosen instead.

Testing

Testing Approach

The testing phase was initially thought to be more of a consistent process throughout the project, the intention was that every time a page, or a feature was completed, it would be tested on how it handles validation, accuracy, and behaves under error conditions. Unfortunately, this was not fully achieved due to several challenges such as, lack of knowledge, limited testing experience, pacing of the workflow, and lack of communication during the early stages of the project.

After the project was more so completed, the testing was approached in two main ways, manual testing and scripted testing using MSTest. Manual testing was focused on checking the website from a user's perspective including navigation, form validation, login behaviour, role-based access, general functionality, and viewing the styling. Automated testing was used to test controller logics and important failure cases.

Build Verification

Build verification was carried out to ensure that the solution could compile and run successfully after changes were made. After pointing out a build error we would try and fix it making sure it would work as intended. The test project was also added to the solution and verified. Although because of the limited time we had for the testing phase, we were not able to fully verify the project, individually we all made an effort to ensure that when making a feature we have to check and verify it works as intended before moving on to creating another feature.

Functional Testing

Functional testing focused on checking whether the main features of the system worked as expected. This included testing logins, registration, unauthorised access, booking-related functionality, attendance behaviour, and role-based access for member, trainer, and management. Other features, such as QR scanning and the AI chatbot, were not fully tested using MsTest due to limited experience with those components. However, they were still checked through manual testing to confirm their general behaviour and functionality.

Challenges Encountered

Several challenges were encountered during testing. One challenge was understanding how to test a large ASP.NET project using MVC with multiple controllers, services, roles, and database-related features. Another issue was identifying the correct users and roles for testing, especially for management, trainer, and member access.

There were also some technical challenges when setting up MsTest, such as understanding helper methods, fake test data, mock objects and in-memory databases. Some tests needed more set up than expected because the controllers depended on services such as UserManager and GymDbContext, which is why using mock objects were required to create fake set ups with fake temporary data for testing. Despite these challenges, the testing process helped identify behaviours and improved understanding of how the system handles both valid and invalid actions,

Conclusion

Summary of Work

And so, it can be stated that the Delta Gym System is a .NET 10 PWA that aids in the management of all facets of a gym. With a desire to go beyond simple membership management, the system merges all operational and managerial concerns into a singular software. With the Delta Gym Software system, one can manage memberships, staff, trainers, classes, bookings, attendances, and other such operations. This is a two-project solution that comprises a REST API and an MVC-based frontend consumer, with inclusion of a shared class library; which some may also say is a "project". This shared library serves as a translation layer between the backend and frontend projects, as the usage of DTOs help to translate expected JSON responses and requests into proper C# objects for seamless transference of data. The separation of these layers aids in independent scalability, and enforces a clean separation of concerns; something that was critical since the beginning of development. The system employs JWT Bearer authentication, API rate limiting, output caching, and pagination. In terms of features, there is a robust QR code generation and scanning system, as well as an AI chat bot, named DeltaBot, with agentic hands-on tooling into the database. Though the codebase lives on GitHub, the system is deployed to Azure, with Azure SQL Server as the database host. Two distinct App Services were created, with one hosting the API and the other hosting the frontend.

The data design partitions the world around it into three large components: Personnel, Infrastructure, and Operations. The Personnel component consists of the single ApplicationUser entity, which inherits and extends Identity's IdentityUser, to deal with the four role types in a single schema with all kinds of gym-related properties (Admin, Staff, Trainer, Member). In terms of Infrastructure, the Branch, Room, and Equipment classes depict physical resources of the gym. The Operational aspect is represented by the entities of Subscription, Payment, Booking, Attendance, Schedule, Session, Class, and Tier that model day-to-day transactions and logistics of the gym operations. Naturally, the relationships between these entities that were implemented via the application database context with the help of the Fluent API were indispensable.

Critical Reflection

Since the start of this endeavor, it was known to all members that the project would consist of technologies and environments that were unfamiliar. As such, it was evident that there would be periods of time spent on learning, and familiarizing oneself with the specific architectures defined for the creation of the software. What was not expected was the disparity between the paces in which members could learn. Perhaps, during the beginning, there should have been more importance placed on this. Truthfully, the first five weeks were not spent well, and for a moment in time, no work, or learning, was being performed towards this project at all due to a mix and mismanagement of priorities. In relation to this, the team did too much waiting. When some members would vanish or simply not be present, the desire to be fair and understanding actually inhibited the progress of the project. In hindsight, choosing to pursue progress rather than waiting around would have aided in project delivery. It is suffice to say that the majority of Delta Software's issues

References

freeCodeCamp.org (2024). *ASP.NET Core MVC Course for Beginners (.NET 9)*. [online] YouTube. Available at: https://www.youtube.com/watch?v=RWXKyslmabs [Accessed 7 Jan. 2025].

GitHub Docs. (2024). *GitHub.com Help Documentation.* [online] Available at: https://docs.github.com/.

Microsoft (2024). *ASP.NET documentation.* [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/aspnet/core.

Microsoft (2021). *Overview of Entity Framework Core - EF Core*. [online] learn.microsoft.com. Available at: https://learn.microsoft.com/en-us/ef/core/.

Microsoft (2024). *Overview of ASP.NET Core MVC*. [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/aspnet/core/mvc/.

Microsoft (2024). *SQL Server technical documentation - SQL Server*. [online] learn.microsoft.com. Available at: https://learn.microsoft.com/en-us/sql/sql-server/.

Microsoft (2024). *Introduction to Identity on ASP.NET Core*. [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity.

Microsoft (2025a). APIs Overview. [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/apis?view=aspnetcore-10.0&WT.mc_id=dotnet-35129-website.

Microsoft (2025b). Tutorial: Add sorting, filtering, and paging - ASP.NET MVC with EF Core. [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/aspnet/core/data/ef-mvc/sort-filter-page?view=aspnetcore-10.0.

Microsoft (2026). Hosting Applications on Azure. [online] learn.microsoft.com Available at: https://learn.microsoft.com/en-us/azure/developer/intro/hosting-apps-on-azure.

Pressman, R.S. and Maxim, B.R. (2020). *Software engineering : a practitioner's approach*. 9th ed. New York, Ny: Mcgraw-Hill Education.